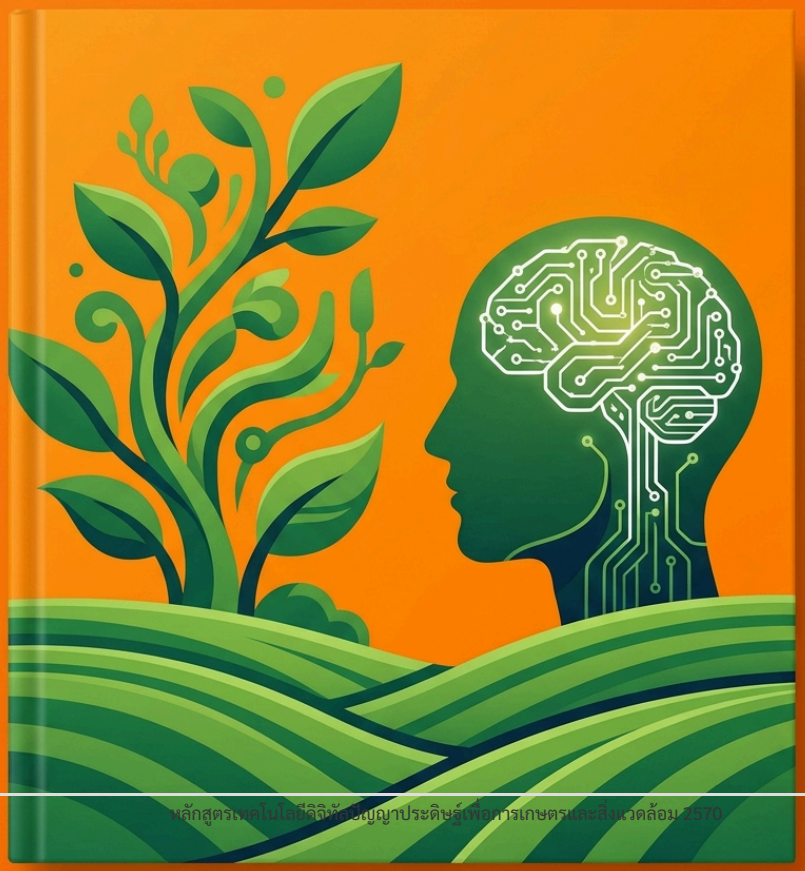
Python xAI

< AI Agent Co-Creator >

Academic Masterbook - Full Implementation Guide



คำนำ (Preface)

หลักสูตร เทคโนโลยีดิจิทัลปัญญาประดิษฐ์เพื่อการเกษตรและสิ่งแวดล้อม และคู่มือปฏิบัติการ "Python x AI 2026: AI-Next Scientist" เล่มนี้ จัดทำขึ้นเพื่อเป็นแนวทางและรากฐานสำคัญสำหรับการบูรณาการความรู้ทางด้านปัญญาประดิษฐ์ (AI) ควบคู่กับศาสตร์ทางการเกษตรสมัยใหม่ โดยมีเป้าหมายเพื่อสร้าง "นักวิทยาศาสตร์รุ่นใหม่" (AI-Next Scientist) ที่ไม่เพียงแต่เขียนโปรแกรมได้ แต่สามารถออกแบบ วางระบบ และกำกับนวัตกรรมแก้ไขปัญหาภาคการเกษตรและสิ่งแวดล้อมได้อย่างเป็นรูปธรรม

เนื้อหาในเล่มครอบคลุมตั้งแต่พื้นฐานการเขียน Python, Machine Learning สำหรับการพยากรณ์ผลผลิต, Deep Learning, ระบบ Computer Vision, และการจำลอง Digital Twin ด้วยมุมมองที่เน้นการลงมือปฏิบัติจริง (Hands-On)

ผู้จัดทำหวังเป็นอย่างยิ่งว่า คู่มือเล่มนี้จะเป็นประโยชน์ต่อผู้เรียน คณาจารย์ และนักวิจัย ที่ต้องการนำปัญญาประดิษฐ์ไปวิวัฒนาการอุตสาหกรรมการเกษตรกรรมของประเทศไทยสืบไป

ผศ.ดร.ชีวะ ทัศนา

กิตติกรรมประกาศ (Acknowledgements)

การจัดทำหลักสูตรและคู่มือปฏิบัติการฉบับนี้สำเร็จลุล่วงไปได้ด้วยดี ขอขอบพระคุณการสนับสนุนเครื่องมือและเทคโนโลยีอันล้ำสมัยจากระบบผู้ช่วยปัญญาประดิษฐ์ **Antigravity**, **Notebook LM** ที่ช่วยกลั่นกรองและสรรค์สร้างเนื้อหาเชิงวิชาการได้อย่างมีประสิทธิภาพ รวมถึงแพลตฟอร์มการจัดการการเรียนรู้ **SciExHub2026** ที่ตระเตรียมโครงสร้างพื้นฐานในการส่งมอบองค์ความรู้สู่ผู้เรียนอย่างเป็นระบบ

ขอขอบพระคุณคณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี ที่ให้การสนับสนุนวิสัยทัศน์ในการพัฒนากำลังคนด้านเทคโนโลยีดิจิทัลและการเกษตรร่วมสมัยเสมอมา

วันและเวลา (Course Schedule)

ระยะเวลา: 12 ชั่วโมง (2 วัน)

ชั่วโมง	หัวข้อ (Topic)	กิจกรรมปฏิบัติการ (Core Activity)
DAY 1	**Foundations & Prediction**	
1-3	บทที่ 1: รากฐานและตรรกะมหัศจรรย์	การใช้ AI ช่วยเขียนตรรกะ & Python Magic
4-6	บทที่ 2: พลังแห่งการพยากรณ์ (ML)	Machine Learning & xAI
DAY 2	**Deep Learning & Reality**	
7-8	บทที่ 3: สมองดิจิทัล	สร้างโมเดล Deep Learning แยกโรคพืช
9-10	บทที่ 4: ดวงตาอัจฉริยะและภาษา AI	OpenCV Masking & Multimodal Chatbot
11	บทที่ 5: Digital Twin Simulation	การจำลองฟาร์มด้วย Pygame Zero
12	บทที่ 6: Agri-Hackathon Stage	บูรณาการนวัตกรรม & Final Pitching

สารบัญ (Table of Contents)

1. **บทที่ 1**: รากฐานและตรรกะมหัศจรรย์ของ AI
2. **บทที่ 2**: พลังแห่งการพยากรณ์ด้วย Machine Learning
3. **บทที่ 3**: สมองดิจิทัล (Deep Learning & Neural Networks)
4. **บทที่ 4**: ดวงตาอัจฉริยะและภาษา AI (Vision & NLP)
5. **บทที่ 5**: Pygame Zero Simulation (The Visualization)
6. **บทที่ 6**: Agri-Hackathon Stage (The Integration)
7. **ภาคผนวก ก.**: คลังรหัสคำสั่งสำหรับผู้เรียน (Student Code Sandbox)
8. **ภาคผนวก ข.** โค้ดเฉลยสำหรับอาจารย์ผู้สอน (TA Solutions)
9. **ภาคผนวก ค.** สคริปต์นำเสนอสำหรับวิทยากร (Instructor Scripts)
10. **ดัชนี**: ดัชนีคำค้นและอภิธานศัพท์ (Glossary & Index)

สารบัญภาพ (List of Figures)

- **ภาพที่ 1**: หน้าจอสภาพแวดล้อม Google Colab (บทที่ 1)
- **ภาพที่ 2**: การสร้างกระบวนการเรียนรู้และแดชบอร์ด (บทที่ 2)
- **ภาพที่ 3**: สถาปัตยกรรมโครงข่ายประสาทเทียมลึก (บทที่ 3)
- **ภาพที่ 4**: แดชบอร์ดสรุปประสิทธิภาพของแบบจำลอง Random Forest (เพิ่มใหม่ - บทที่ 2)
- **ภาพที่ 5**: ระบบตรวจจับโรคด้วยค่าสี HSV (บทที่ 4)
- **ภาพที่ 6**: การวิเคราะห์ความรู้สึกและตอบโต้ Chatbot (บทที่ 4)
- **ภาพที่ 7**: แผนผังการจำลอง DigitalTwin (เพิ่มใหม่ - บทที่ 5)

สารบัญตาราง (List of Tables)

- **ตารางที่ 1**: โครงสร้างและกำหนดการอบรม (Course Schedule)
- **ตารางที่ 2**: ภารกิจการเขียนโค้ดตามระดับของผู้เรียน (หมวดที่ 1)

บทที่ 1: AI Co-Pilot Mastery

ระยะเวลา: ชั่วโมงที่ 1 - 3 (3 ชั่วโมง) **หัวใจสำคัญ**: เปลี่ยนจากการ "เขียนโค้ด" ไปสู่การ "กำกับ AI" เพื่อสร้างตรรกะนวัตกรรม

ชั่วโมงที่ 1: ก้าวแรกสู่ Cloud Lab

ทำความรู้จักกับ Google Colab ห้องทดลองที่ทรงพลังที่สุดบนระบบ Cloud

Copy-Paste Lab: ตรวจสอบระบบ

รันโค้ดชุดนี้เพื่อดูว่า Colab ของคุณพร้อมทำงานหรือไม่

.py / .ipynb

```
import sys
print(f"Python Version: {sys.version}")
print("Cloud Lab Initialized: Ready to Code!")
```

Minecraft Spotlight: พิกัด 3 มิติ (X, Y, Z)

ใน Minecraft โลกประกอบด้วยพิกัด **X (ซ้าย-ขวา)**, **Y (ความสูง)** และ **Z (ลึก)**

- การขยับ Agent ในเกมด้วย Python คือการเปลี่ยนค่า `pos(x, y, z)`
- เช่นเดียวกับอาเรย์ที่เราจะเรียน: `farm[x][y]` คือการระบุตำแหน่งดินในฟาร์ม!

AI Co-Pilot: เพื่อนคู่คิดคนใหม่ (Real-Time Debugging)

ตั้งแต่วินาทีนี้ไป คุณจะไม่ต้องนั่งงมกับ Error คนเดียวอีกต่อไป! หากโค้ดรันไม่ได้ ให้ลอง Copy Error ไปถาม AI ด้วยประโยคนี้:

> *"ฉันเป็นนักเรียนที่กำลังเรียน Python ใน Google Colab และเจอข้อผิดพลาดนี้: [แปะ Error] ช่วยอธิบายว่ามันเกิดจากอะไร และแนะนำวิธีแก้โค้ดทีละขั้นตอนให้หน่อย"*

ชั่วโมงที่ 2: ศิลปะการสั่งงาน AI (S.C.I.E.N.T.I.S.T. Prompting)

เพื่อให้ AI ช่วยคุณสร้างนวัตกรรมที่แม่นยำ เราจะใช้สูตรการสั่งงานระดับโลกที่ครอบคลุมทุกมิติ:

อักษร	ความหมาย	รายละเอียดการสั่ง
S	Situation	บอกสถานะปัจจุบัน ("ฉันกำลังทำโปรเจกต์แยกโรคพืช...")
C	Context	ให้ข้อมูลแวดล้อม ("โดยใช้ข้อมูลอุณหภูมิและความชื้น...")
I	Input	ใส่ข้อมูลหรือโค้ดที่มี ("นี่คือโค้ดที่ฉันมี [แปะโค้ด]...")
E	Expectation	บอกผลลัพธ์ที่อยากได้ ("ฉันอยากให้ AI ตรวจสอบว่าพืชเสี่ยงเป็นโรคไหม...")
N	Nuance	ระบุรายละเอียดปลีกย่อย ("ขอคำอธิบายที่เด็ก ม.ต้น เข้าใจได้...")
T	Target	ระบุกลุ่มเป้าหมายหรือวัตถุประสงค์ ("เพื่อใช้ในการพรีเซนต์โครงงาน...")
I	Insight	ขอคำแนะนำเชิงลึก ("แนะนำแนวทางการพัฒนาต่อยอดให้ด้วย...")

S	Style	กำหนดรูปแบบคำตอบ ("แสดงผลเป็นข้อๆ หรือเป็นตาราง...")
T	Test	ให้ AI ช่วยตรวจสอบ ("ช่วยเขียน Test Case เพื่อเช็คโค้ดนี้ให้หน่อย...")

AI จะฉลาดได้ต้องมี "ตรรกะ" (Logic) เราจะสั่งให้คอมพิวเตอร์ "คิด" และ "ทำ" ตามเงื่อนไข

Copy-Paste Lab: ภารกิจทักทาย (Junior/Senior)

สร้างตัวแปรเพื่อเก็บข้อมูลและแสดงผลลัพธ์

.py / .ipynb

```
# ระบบบันทึกชื่อนักนวัตกรรม
name = input("ระบุชื่อรหัสนักวิทย์ของคุณ: ")
print(f"ยินดีต้อนรับคุณ {name} เข้าสู่ปี 2026!")

# ระบบตรวจสอบอุณหภูมิฟาร์มอัจฉริยะ
temp = 38
if temp > 35:
    print("   แจ้งเตือน: อุณหภูมิสูงเกินไป! เปิดพัดลม...")
else:
    print("   สถานะ: อุณหภูมิปกติ")
```

ชั่วโมงที่ 3: ระบบคัดกรองข้อมูล (Data Filtering)

เรียนรู้การทำงานกับข้อมูลหลายๆ ชุด (Lists) และการวนลูป (Loops)

Copy-Paste Lab: เซนเซอร์พยากรณ์

.py / .ipynb

```
# ข้อมูลความชื้นจาก 5 จุดในฟาร์ม
moisture_levels = [45, 32, 55, 28, 40]

for m in moisture_levels:
    status = "ปกติ" if m >= 40 else "ต่ำเกินไป (ต้องรดน้ำ)"
    print(f"จุดตรวจที่ระดับ {m}%: สถานะ {status}")
```

Checklist ท้ายบท

- ☐ รันโค้ดใน Colab สำเร็จ
- ☐ เข้าใจการใช้ `if-else` ในการตัดสินใจ
- ☐ สามารถรัน `for loop` เพื่อวนลูปอ่านค่าข้อมูลได้

Minecraft Challenge: หุ่นยนต์เก็บเกี่ยว (Automated Harvesting)

ลองใช้ความรู้เรื่อง **Loop** ในบทนี้ สั่งให้ Agent เดินเก็บเกี่ยวพืชในรัศมี 10 บล็อกรอบตัว:

.py / .ipynb

```
# สั่งหุ่นยนต์เดินเก็บของในพื้นที่
for length in range(10):
    agent.destroy(FORWARD)
    agent.move(FORWARD, 1)
```

Challenge (คำถามท้าทาย)

หากต้องการให้ระบบแจ้งเตือน "วิกฤต!" เมื่อความชื้นต่ำกว่า **20%** คุณต้องแก้ไขเงื่อนไขในโค้ดบรรทัดไหน?

บทที่ 2: พลังแห่งการพยากรณ์

ระยะเวลา: ชั่วโมงที่ 4 - 6 (3 ชั่วโมง) **หัวใจสำคัญ**: การใช้ Machine Learning ทายอนาคต และอธิบายเหตุผล (xAI)

ชั่วโมงที่ 4: จากข้อมูลสู่ความรู้ (Machine Learning)

เรียนรู้ความแตกต่างระหว่าง "การเขียนโปรแกรมปกติ" และ "การสอน AI"

Copy-Paste Lab: ตรวจสอบความถูกต้องของข้อมูล

ข้อมูลในโลกจริงมักจะมีช่องโหว่ เราต้องทำความสะอาดก่อนเสมอ

.py / .ipynb

```
import pandas as pd
import numpy as np

# จำลองข้อมูลจากฟาร์ม 5 จุด
data = {'ความชื้น': [45, 32, np.nan, 28, 40], 'ปุ๋ย': [10, 15, 12, np.nan, 18]}
df = pd.DataFrame(data)

# ซ่อมข้อมูลแหว่งด้วยค่าเฉลี่ย (Mean)
df.fillna(df.mean(), inplace=True)
print("   ข้อมูลถูกซ่อมเรียบร้อยแล้ว!")
print(df)
```

ชั่วโมงที่ 5: การสอน AI ด้วย Random Forest

เราจะใช้โมเดล "ป่าสุ่มร้อยต้น" มาทำงานแทนเรา

ภาพที่ 4: แดชบอร์ดสรุปประสิทธิภาพของแบบจำลอง Random Forest

Copy-Paste Lab: สร้างและรัน AI Model

.py / .ipynb

```
from sklearn.ensemble import RandomForestClassifier

# 1. ข้อมูลตัวอย่าง [ดินดี, น้ำชุ่ม, แสงพอ]
X = [[1, 1, 1], [0, 0, 0], [1, 0, 1], [0, 1, 0]]
y = ['โตดี', 'เฉา', 'พอใช้', 'พอใช้']

# 2. สร้างโมเดลและฝึกสอน
model = RandomForestClassifier(n_estimators=100)
model.fit(X, y)

# 3. ลองให้ AI ทายค่าใหม่!
test_farm = [[1, 1, 0]] # ดินดี, น้ำชุ่ม, แสงน้อย
prediction = model.predict(test_farm)
print(f"   AI ทายว่า: {prediction[0]}")
```

ชั่วโมงที่ 6: เจาะลึกสมอง AI (xAI)

ทำไม AI ถึงตัดสินใจแบบนั้น? เราจะส่องดูค่าน้ำหนักความสำคัญ (Feature Importance)

Copy-Paste Lab: ค้นหาความลับของข้อมูล

.py / .ipynb

```
# ค้นหาว่าตัวแปรไหนมีผลต่อการตัดสินใจที่สุด
importance = model.feature_importances_
features = ['คุณภาพดิน', 'ปริมาณน้ำ', 'แสงแดด']

for f, imp in zip(features, importance):
    print(f"   {f}: มีความสำคัญ {imp*100:.2f}%")
```

Checklist ท้ายบท

- ☐ สามารถสร้างโมเดล Machine Learning และดูเหตุผล (xAI) ได้
- ☐ เข้าใจการทำงานของ Random Forest
- ☐ สามารถดึงความสำคัญของฟีเจอร์ (Feature Importance) ออกมาได้

Minecraft Challenge: หุ่นยนต์เก็บเกี่ยว (Automated Harvesting)

ลองใช้ความรู้เรื่อง **Loop** ในบทนี้ สั่งให้ Agent เดินเก็บเกี่ยวพืชในรัศมี 10 บล็อกรอบตัว:

.py / .ipynb

```
# สั่งหุ่นยนต์เดินเก็บของในพื้นที่
for length in range(10):
    agent.destroy(FORWARD)
    agent.move(FORWARD, 1)
```

Challenge (คำถามท้าทาย)

จากผลลัพธ์ของ xAI ปัจจัยไหน (ดิน, น้ำ, หรือแสง) ที่มีน้ำหนักความสำคัญต่อพืชชนิดนี้มากที่สุด?

บทที่ 3: สมองดิจิทัล (Deep Learning)

ระยะเวลา: ชั่วโมงที่ 7 - 8 (2 ชั่วโมง) **หัวใจสำคัญ**: การเลียนแบบเซลล์ประสาทมนุษย์ เพื่อแยกแยะโรคพืช

ชั่วโมงที่ 7: โครงสร้างเซลล์ประสาทเทียม (Neural Networks)

ทำความเข้าใจเลเยอร์ต่างๆ ของสมอง AI (Input, ReLU, Softmax)

Copy-Paste Lab: การเลือกฟังก์ชันกระตุ้น (ReLU)

.py / .ipynb

```
# ReLU คือคนคัดกรอง: ถ้าเป็นค่าลบให้เป็น 0 ถ้าเป็นบวกให้ผ่านได้
sample_values = [-5, 12, -2, 8]
relu_output = [max(0, x) for x in sample_values]
print(f"ผลลัพธ์จาก ReLU สำหรับ {sample_values} คือ {relu_output}")
```

ชั่วโมงที่ 8: ภารกิจสร้างโมเดลแยกโรคพืช (CNN)

เราจะสร้าง AI ที่ "มองเห็น" และแยกโรคพืชได้โดยใช้ภาพ

Copy-Paste Lab: โครงสร้างโมเดลจำแนกภาพ

.py / .ipynb

```
# จำลองการสร้าง AI สมองกล 4 เลเยอร์
def build_agri_brain():
    print("สร้าง Input Layer... (รับภาพใบไม้)")
    print("เพิ่ม ReLU Layer... (กรองข้อมูล)")
    print("เพิ่ม Dropout Layer (0.2)... (ป้องกันการจำผิด)")
    print("เพิ่ม Softmax Layer... (ทำนายว่าเป็นโรคอะไร)")

build_agri_brain()
print("   สมองดิจิทัลพร้อมเรียนรู้แล้ว!")
```

Checklist ท้ายบท

- [ ] เข้าใจหน้าที่ของปุ่ม ReLU ในการกรองข้อมูล
- [ ] เข้าใจการใช้ Dropout เพื่อป้องกันการจำคำตอบ (Overfitting)
- [ ] สามารถอธิบายโครงสร้างพื้นฐานของ Neural Networks ได้

Challenge (คำถามท้าทาย)

หาก AI ทายว่าเป็น "โรคใบไหม้" 80% และ "ปกติ" 20% ฟังก์ชันใดทำหน้าที่สรุปผลตัวเลขเป็นเปอร์เซ็นต์? (Softmax หรือ ReLU)

บทที่ 4: ดวงตาอัจฉริยะและภาษา AI (CV & NLP)

ระยะเวลา: ชั่วโมงที่ 9 - 10 (2 ชั่วโมงเต็ม) **หัวใจสำคัญ**: ก้าวข้ามจาก "ข้อมูล" สู่ "การรับรู้" (Perception) ทั้งภาพและภาษา

ในบทนี้ เราจะกระชับเนื้อหาให้เข้มข้นและรวดเร็ว เพื่อให้นักเรียนสามารถนำ AI ไปประยุกต์ใช้ในการตรวจสุขภาพพืชและสั่งงานด้วยเสียงได้ทันทีครับ

พาร์ทที่ 1: Computer Vision (60 นาที)

แบ่งเป็น 2 ช่วงสั้นๆ เพื่อให้เห็นผลลัพธ์ที่เป็นรูปธรรม

ช่วงที่ 1: ตรรกะระดับพิกเซลและสี (30 นาที)

เรียนรู้การทำงานของ **HSV Masking** ที่แม่นยำกว่า RGB

> *[!TIP] ใช้โฟลเดอร์* `workshop_code/python_scripts/ch04_cv_nlp/ex01` *และ* `ex02`

ช่วงที่ 2: การตรวจจับวัตถุด้วยโมเดลสำเร็จรูป (30 นาที)

การใช้ **Haar Cascades** เพื่อตรวจหาใบพืชหรือวัตถุในฟาร์ม

.py / .ipynb

```python
# ตัวอย่างการโหลดโมเดล (Boilerplate)
import cv2
face_cascade = cv2.CascadeClassifier('models/haarcascade_agri.xml')
# ... ตรวจจับและวาดกรอบสี่เหลี่ยม ...
```

พาร์ทที่ 2: Natural Language Processing (60 นาที)

เปลี่ยนข้อความเป็นคำสั่งงานเชิงลึก

ช่วงที่ 3: Intent & Sentiment (30 นาที)

เข้าใจว่า AI "เดาใจ" ผู้ใช้อย่างไร ผ่านคีย์เวิร์ดและคะแนนความรู้สึก (Sentiment)

> *[!TIP] ใช้โฟลเดอร์* `workshop_code/python_scripts/ch04_cv_nlp/ex03` *และ* `ex04`

ช่วงที่ 4: Agri-Bot Liaison (30 นาที)

ประกอบร่างเป็นระบบแชทบอทถาม-ตอบเรื่องการเกษตรที่โต้ตอบได้จริง

> *[!TIP] ใช้โฟลเดอร์* `workshop_code/python_scripts/ch04_cv_nlp/ex05`

Checklist ท้ายบท

- [ ] สามารถแยกเฉดสีน้ำตาล (โรคพืช) ออกจากใบไม้ได้สำเร็จ
- [ ] เข้าใจการทำงานของระบบตัดสินใจผ่าน Intents
- [ ] สามารถประยุกต์ใช้โมเดลสำเร็จรูป (Inference) ในการตรวจจับวัตถุได้

> *[!IMPORTANT] **คำแนะนำสำหรับวิทยากร**: หากเวลาเหลือน้อย ให้เน้นที่การรันโค้ดสำเร็จรูปใน `ta_solutions` เพื่อให้เด็กๆ ได้เห็นผลลัพธ์ที่น่าตื่นเต้น (Wow Factor) ก่อนจบวัน*

บทที่ 5: Digital Twin Simulation (2D & 3D)

ระยะเวลา: ชั่วโมงที่ 11 (1 ชั่วโมง) **หัวใจสำคัญ**: การเชื่อมโยงโลกเสมือน (Software) เข้ากับตรรกะ AI เพื่อจำลองการทำงานก่อนลงพื้นที่จริง

ในบทนี้ เราจะเปลี่ยนจากการ "เขียนเกม" ทั่วไป มาเป็นการสร้าง **Digital Twin Dashboard** ซึ่งเป็นเทคโนโลยีที่นักวิทยาศาสตร์ใช้จำลองฟาร์มหรือโรงงานในคอมพิวเตอร์ เพื่อทดสอบโค้ด AI ของเราว่าทำงานได้ถูกต้องไหม ก่อนจะเอาไปใส่ในหุ่นยนต์จริงๆ ครับ

ตอนที่ 1: 2D Dashboard ด้วย Pygame Zero

การเริ่มต้นเขียนโปรแกรมจำลองด้วย **Pygame Zero (PyZero)** นั้นง่ายและเห็นผลเร็ว เหมาะสำหรับสร้างหน้านักแสดง (Dashboard) เพื่อดูสถานะของฟาร์ม

โครงสร้างเวทมนตร์ของ PyZero (3 Magic Parts)

1. **Setup**: ตั้งค่าหน้าจอ
2. **Draw**: วาดภาพลงบนจอ (เหมือนพู่กันวิเศษ)
3. **Update**: ตรรกะการขยับและความคิด (Logic)

Copy-Paste Lab: Agri-Monitoring System

ลองคัดลอกโค้ดนี้ไปรัน เพื่อสร้างระบบติดตามรถเก็บเกี่ยวอัตโนมัติ

.py / .ipynb

```
import pgzrun

# 1. ตั้งค่าหน้าจอ
WIDTH = 800
HEIGHT = 600

# สร้างตัวละคร (อย่าลืมใส่รูป harvester.png ในโฟลเดอร์ images)
harvester = Actor('harvester')
harvester.pos = 400, 300

# 2. ส่วนของการวาดผลลัพธ์
def draw():
    screen.clear()
    screen.fill((34, 139, 34)) # สีเขียวฟาร์ม
    harvester.draw()
    screen.draw.text("Agri-AI Simulation Mode", (20, 20), color="white", fontsize=30)

# 3. ตรรกะการควบคุม (AI Logic Simulation)
def update():
    # จำลองว่าถ้า AI ตรวจพบพืชพร้อมเก็บ (กด space) ให้รถขยับ
    if keyboard.space:
        harvester.x += 2

pgzrun.go()
```

ตอนที่ 2: 3D Digital Twin ด้วย Minecraft Education

เมื่อเข้าใจตรรกะใน 2 มิติแล้ว เราจะอัปเกรดเข้าสู่โลก 3 มิติ เพื่อทดสอบว่า "โค้ด" ของเราสามารถสั่งงานหุ่นยนต์ (Agent) ให้ทำงานในพื้นที่จริงได้อย่างไร

เริ่มต้นใช้งาน Code Builder

1. เปิดเกม **Minecraft Education**

2. กดปุ่ม **"C"** เพื่อเปิด Code Builder
3. เลือกภาษา **"Python"**

Lab: หุ่นยนต์เสกไร่ข้าว (The Instant Farm)

Mission: ใช้ Loop เพื่อสั่งให้ Agent ปลูกข้าว 10x10 บล็อก

.py / .ipynb

```
# สั่งให้หุ่นยนต์เดินและปลูกพืชอัตโนมัติ (Nested Loop)
for i in range(10):
    for j in range(10):
        agent.place(FORWARD, 1) # วางบล็อก
        agent.move(FORWARD, 1) # เดินหน้า
    agent.move(LEFT, 1) # ขยับแถวใหม่
    agent.turn(LEFT)
```

Multimodal Logic: เชื่อมโยงความรู้

ในฐานะ **AI-Next Scientist** คุณสามารถรวมทุกบทที่เรียนมาเข้าด้วยกัน:

1. **บทที่ 2-3 (Brain)**: ใช้ ML ทายว่าฝนจะตกไหม
2. **บทที่ 4 (Vision)**: ใช้ CV มองเห็นพื้นที่ใน Minecraft
3. **บทที่ 5 (Body)**: สั่งให้ Agent ขยับตามคำสั่งที่ Brain และ Vision ประมวลผลได้!

Checklist ท้ายบท

- [ ] เข้าใจความหมายของ Digital Twin และการจำลอง (Simulation)
- [ ] สามารถขยับตัวละครใน 2D ได้สำเร็จ
- [ ] สามารถสั่ง Agent ใน Minecraft ให้ทำงานตามตรรกะ Loop ได้

> *[!IMPORTANT] **พี่ยักษ์สรุปให้**: การจำลองในโลก Digital ช่วยลดความเสี่ยงและความผิดพลาดในโลกจริง (Physics World) นี่คือพลังของนักวิทยาศาสตร์ยุค 2026 ครับ!*

บทที่ 6: Agri-Hackathon Stage

ระยะเวลา: ชั่วโมงที่ 12 (1 ชั่วโมง) **หัวใจสำคัญ**: การบูรณาการความรู้ทั้งหมดและนำเสนอผลงาน

ชั่วโมงที่ 12: การสร้างนวัตกรรมต้นแบบและนำเสนอผลงาน

รวบรวมทุกชิ้นส่วน (Logic + ML + Vision + NLP) มาแก้ปัญหาคนเกษตร

Copy-Paste Lab: โครงสร้างโปรเจกต์ต้นแบบ

.py / .ipynb

```python
# ระบบ AI ควบคุมฟาร์มอัจฉริยะแบบเบ็ดเสร็จ
def agri_startup_logic(sensor_input, ai_prediction):
    if sensor_input > 40:
        return "ACTIVATE_COOLING"

    if ai_prediction == 'READY':
        return "DEPLOY_HARVESTER"

    return "MONITORING_MODE"

print(f"   แผนธุรกิจของคุณ: {agri_startup_logic(42, 'READY')}")
```

(ต่อ) การนำเสนอผลงาน (The 3-Minute Pitch)

เล่าเรื่องราวความสำเร็จของคุณให้น่าสนใจ!

โครงสร้างการเล่าเรื่อง (Pitch Template)

1. **The Pain (ปัญหา)**: "รู้ไหมครับ... เกษตรกรแถวบ้านผมต้องเจอกับปัญหาเพลี้ยลงทุกปี..."
2. **The AI Solution (การแก้ปัญหา)**: "พวกเราจึงสร้าง AI ที่มองเห็นเพลี้ยผ่านกล้อง (CV) และทายเวลาเพลี้ยระบาด (ML)..."
3. **The Impact (ผลลัพธ์)**: "นวัตกรรมนี้จะลดการใช้สารเคมีได้ 40% และเพิ่มรายได้ 3 เท่า!"

Checklist ท้ายบท

- [ ] บูรณาการ Logic และ AI ได้เรียบร้อย
- [ ] มีผลลัพธ์การรันจาก Colab ที่แสดงผลความถูกต้อง
- [ ] เตรียมสไลด์นำเสนอ (Pitch) เสร็จทันเวลา

ยินดีด้วยคุณคือ Next-Gen Scientist 2026!

คุณได้ก้าวข้ามขีดจำกัดจากผู้ใช้ มาเป็น **"ผู้สร้าง"** เรียบร้อยแล้ว ขอให้นำความรู้นี้ไปต่อยอดเพื่อสร้างสรรค์โลกที่น่าอยู่ขึ้นด้วย AI นะครับ!

ภาคผนวก ก. คลังรหัสคำสั่งสำหรับผู้เรียน (Student Code Sandbox)

ส่วนนี้นำเสนอชุดคำสั่งที่เตรียมไว้ให้นักเรียน โดยแบ่งออกเป็น 2 ชนิดอ้างอิงตามลักษณะโจทย์:

1. **แฟ้มสคริปต์ (Python Script - .py)**: โค้ดรันโปรแกรมระบบจำลอง โลคอลหรือ Pygame
2. **สมุดจด (Jupyter Notebook - .ipynb)**: สำหรับงาน Data Science รันบน Colab ซึ่งสามารถโหลดใช้งานได้จากโฟลเดอร์ `workshop_code/notebooks/`

ก.1 คลังรหัสคำสั่ง Python Script (.py)

อ้างอิงรหัสคำสั่งและการนำไปใช้งานจากแบบฝึกหัดในห้องเรียน (Hands-on Labs)

ส่วนเสริม: ch01_foundations / ex01_check_system.py

.py / .ipynb

```python
import sys
print(f"Python Version: {sys.version}")
print("Cloud Lab Initialized: Ready to Code!")
```

ส่วนเสริม: ch01_foundations / ex01_hello.py

.py / .ipynb

```python
# ex01_hello.py
#    ระดับ: Beginner
print("ยินดีต้อนรับสู่ Python x AI 2026!") # พิมพ์ข้อความทักทาย
print("--- [Day 1: Start] ---")
```

ส่วนเสริม: ch01_foundations / ex02_greeting_bot.py

.py / .ipynb

```python
name = input("ระบุชื่อรหัสนักวิทย์ของคุณ: ")
print(f"ยินดีตอนรับคุณ {name} เข้าสู่ปี 2026!")

temp = 38
if temp > 35:
    print("   แจ้งเตือน: อุณหภูมิสูงเกินไป! เปิดพัดลม...")
else:
    print("   สถานะ: อุณหภูมิปกติ")
```

ส่วนเสริม: ch01_foundations / ex02_vars_math.py

.py / .ipynb

```python
# ex02_vars_math.py
#    ระดับ: Beginner
temp = 32.5
humidity = 65
soil_moist = 45

# คำนวณค่าเฉลี่ยเซนเซอร์
avg = (temp + humidity + soil_moist) / 3
print(f"ค่าเฉลี่ยเซนเซอร์รวม: {avg:.2f}")
```

ส่วนเสริม: ch01_foundations / ex03_input_bot.py

.py / .ipynb

```
# ex03_input_bot.py
#    ระดับ: Beginner
name = input("สวัสดี! คุณชื่ออะไร? ")
print(f"   ยินดีต้อนรับคุณ {name} เข้าสู่แล็บ AI!")

age = int(input("คุณอายุเท่าไหร่? "))
if age < 15:
    print("คุณคือเยาวชนรุ่นใหม่ (Junior Explorer)   ")
else:
    print("คุณคือหัวหอกนวัตกรรม (Senior Innovator)   ")
```

ส่วนเสริม: ch01_foundations / ex04_agri_logic.py

.py / .ipynb

```
# ex04_agri_logic.py
#    ระดับ: Beginner
temp = float(input("ระบุอุณหภูมิฟาร์ม (°C): "))

if temp > 38:
    print("   อันตราย: ความร้อนวิกฤต! เปิดระบบพ่นหมอก...")
elif temp > 35:
    print("   แจ้งเตือน: อากาศร้อน เริ่มเปิดพัดลมระบายอากาศ")
else:
    print("   สถานะ: อุณหภูมิปกติ")
```

ส่วนเสริม: ch01_foundations / ex05_loop_sensor.py

.py / .ipynb

```
moisture_levels = [45, 32, 55, 28, 40]

for m in moisture_levels:
    status = "ปกติ" if m >= 40 else "ต่ำเกินไป (ต้องรดน้ำ)"
    print(f"จุดตรวจที่ระดับ {m}%: สถานะ {status}")
```

ส่วนเสริม: ch01_foundations / ex06_list_data.py

.py / .ipynb

```
# ex06_list_data.py
#    ระดับ: Intermediate
#    ภารกิจ: เก็บสถิติอุณหภูมิรายวันและหาค่าสูงสุด

temp_history = [32.5, 34.1, 38.2, 31.5, 35.0, 36.4, 30.2]

# คำนวณค่าสูงสุดและค่าเฉลี่ย
max_temp = max(temp_history)
avg_temp = sum(temp_history) / len(temp_history)

print(f"   อุณหภูมิสูงสุดในสัปดาห์: {max_temp}°C")
print(f"   อุณหภูมิเฉลี่ย: {avg_temp:.2f}°C")
```

ส่วนเสริม: ch01_foundations / ex07_smart_fan.py

.py / .ipynb

```python
# ex07_smart_fan.py
#    ระดับ: Advanced
#    ภารกิจ: รวมตรรกะทั้งหมดเข้าด้วยกัน (Integrated Fan Control)

def check_fan_status(temp, hum):
    if temp > 35 or hum > 80:
        return "เปิดพัดลมแรงสุด (Level 3)"
    elif temp > 30:
        return "เปิดพัดลมอ่อน (Level 1)"
    else:
        return "ปิดพัดลม"

# จำลองการรับค่าจากเซนเซอร์จริง
current_temp = 36.5
current_hum = 82.0

status = check_fan_status(current_temp, current_hum)
print(f"   ตรวจพบ Temp: {current_temp}, Humidity: {current_hum}")
print(f"   การตัดสินใจของระบบ: {status}")
```

ส่วนเสริม: ch02_ml_intelligence / ex01_csv_load.py

.py / .ipynb

```python
# ex01_csv_load.py
#    ระดับ: Beginner
import pandas as pd

# 1. โหลดข้อมูลจาก CSV (ต้องมีไฟล์ crop_data_v2.csv)
df = pd.read_csv('../../datasets/crop_data_v2.csv')

# 2. ดูข้อมูล 5 แถวแรก
print("--- [Loaded Agriculture Data] ---")
print(df.head())

# 3. ดูขนาดของข้อมูล
print(f"\n   ขนาดข้อมูล: {df.shape[0]} แถว, {df.shape[1]} คอลัมน์")
```

ส่วนเสริม: ch02_ml_intelligence / ex02_trend_plot.py

.py / .ipynb

```python
# ex02_trend_plot.py
#    ระดับ: Intermediate
import pandas as pd
import matplotlib.pyplot as plt

# 1. โหลดข้อมูล
df = pd.read_csv('../../datasets/crop_data_v2.csv')

# 2. พล็อตความสัมพันธ์ระหว่างอุณหภูมิและผลผลิต
plt.scatter(df['temp'], df['yield_weight'], color='orange')
plt.title('Relationship: Temperature vs Yield Weight')
plt.xlabel('Temperature (°C)')
plt.ylabel('Yield Weight (kg)')
plt.grid(True)

print("   กำลังแสดงกราฟแนวโน้ม... (ต้องการสภาพแวดล้อมที่มี GUI)")
# plt.show() # รันบรรทัดนี้เพื่อดูภาพ
```

ส่วนเสริม: ch02_ml_intelligence / ex03_cleaner.py

.py / .ipynb

```python
import pandas as pd
import numpy as np

data = {'ความชื้น': [45, 32, np.nan, 28, 40], 'ปุ๋ย': [10, 15, 12, np.nan, 18]}
df = pd.DataFrame(data)

df.fillna(df.mean(), inplace=True)
print("   ข้อมูลถูกซ่อมเรียบร้อยแล้ว!")
print(df)
```

ส่วนเสริม: ch02_ml_intelligence / ex04_data_processing.py

.py / .ipynb

```
# ex04_data_processing.py
#    ระดับ: Senior (ม.ปลาย)
#    ภารกิจ: เติมข้อมูลที่หายไปด้วยค่าเฉลี่ย (Data Imputation)

import pandas as pd
import numpy as np

# 1. โหลดข้อมูลจำลอง (หรือโหลดจาก CSV)
data = {
    'temperature': [32, 38, np.nan, 22, 40],
    'soil_moisture': [45, 32, 55, np.nan, 40]
}
df = pd.DataFrame(data)

print("--- [Before: Data with NaNs] ---")
print(df)

# 2. เติมค่าว่างด้วยค่าเฉลี่ยของคอลัมน์นั้นๆ
df_clean = df.fillna(df.mean())

print("\n--- [After: Clean Data] ---")
print(df_clean)
```

ส่วนเสริม: ch02_ml_intelligence / ex04_yield_pred.py

.py / .ipynb

```
# ex04_yield_pred.py
#    ระดับ: Intermediate
import pandas as pd
from sklearn.linear_model import LinearRegression

# 1. โหลดข้อมูล
df = pd.read_csv('../../datasets/crop_data_v2.csv')

# 2. เตรียม X (Input: แสง) และ y (Output: น้ำหนัก)
X = df[['light']]
y = df['yield_weight']

# 3. สร้างและฝึกฝนโมเดล
model = LinearRegression()
model.fit(X, y)

# 4. พยากรณ์ถ้านักเรียนให้แสง 16 ชั่วโมง
future_light = [[16]]
pred_weight = model.predict(future_light)

print(f"   พยากรณ์: ถ้าให้แสง {future_light[0][0]} ชม. จะได้รับผลผลิต {pred_weight[0]:.2f} kg")
```

ส่วนเสริม: ch02_ml_intelligence / ex05_random_forest.py

.py / .ipynb

```
# ex05_random_forest.py
#    ระดับ: Senior (ม.ปลาย)
#    ภารกิจ: รถเก็บเกี่ยวอัจฉริยะ (Smart Harvester) ทายผลผลิตจากข้อมูลฟาร์ม

import pandas as pd
from sklearn.ensemble import RandomForestClassifier

# 1. โหลดข้อมูลจำลอง (ใช้ข้อมูลที่เราเตรียมไว้ใน datasets)
# (หรือทดลองสร้าง Mock Data เล็กๆ เพื่อเทส)
X = [[32, 65, 45], [38, 70, 32], [25, 45, 55], [22, 40, 28]]
y = ['READY', 'READY', 'LOW_YIELD', 'LOW_YIELD']

# 2. สร้างโมเดล (ป่าสุ่ม 100 ต้น)
model = RandomForestClassifier(n_estimators=100)
model.fit(X, y)

# 3. ลองทายผลผลิตจากค่าเซนเซอร์ใหม่
new_sensor = [[35, 75, 35]] # อุณหภูมิ, ความชื้นอากาศ, ความชื้นดิน
prediction = model.predict(new_sensor)

print(f"   AI พยากรณ์สถานะ: {prediction[0]}")
```

ส่วนเสริม: ch02_ml_intelligence / ex05_rf_classifier.py

.py / .ipynb

```
# ex05_rf_classifier.py
#    ระดับ: Intermediate
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

# 1. โหลดข้อมูลเต็มรูปแบบ
df = pd.read_csv('../../datasets/crop_data_v2.csv')

# 2. คัดเลือก X (ปัจจัย) และ y (สถานะ)
X = df[['temp', 'humidity', 'soil_moist', 'fert', 'light']]
y = df['label']

# 3. แบ่งข้อมูล (Train/Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. สร้างและฝึกฝนโมเดล
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

# 5. ทายผลจากข้อมูลทดสอบ
acc = model.score(X_test, y_test)
print(f"   ความแม่นยำของโมเดล (Accuracy): {acc * 100:.2f}%")
```

ส่วนเสริม: ch02_ml_intelligence / ex06_compare_models.py

.py / .ipynb

```
# ex06_compare_models.py
#    ระดับ: Advanced
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

# 1. โหลดข้อมูล
df = pd.read_csv('../../datasets/crop_data_v2.csv')
X = df[['temp', 'humidity', 'soil_moist', 'fert', 'light']]
y = df['label']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 2. เปรียบเทียบ 2 โมเดล
dt = DecisionTreeClassifier()
rf = RandomForestClassifier()

dt.fit(X_train, y_train)
rf.fit(X_train, y_train)

print(f"   Decision Tree Score: {dt.score(X_test, y_test)*100:.2f}%")
print(f"   Random Forest Score: {rf.score(X_test, y_test)*100:.2f}%")
```

ส่วนเสริม: ch02_ml_intelligence / ex06_xai_importance.py

.py / .ipynb

```
# ex06_xai_importance.py
#    ระดับ: Senior (ม.ปลาย)
#    ภารกิจ: ส่องสมอง AI ด้วย xAI (Explainable AI)

import pandas as pd
from sklearn.ensemble import RandomForestClassifier

# 1. ข้อมูลตัวอย่าง
data = {
    'temp': [32, 38, 25, 22, 40],
    'soil': [45, 32, 55, 28, 40],
    'light': [12, 14, 8, 6, 15],
    'label': [1, 1, 0, 0, 1] # 1: โตดี, 0: โตช้า
}
df = pd.DataFrame(data)

# 2. ฝึกสอนโมเดล
X = df.drop('label', axis=1)
y = df['label']
model = RandomForestClassifier(n_estimators=100)
model.fit(X, y)

# 3. ดูความสำคัญของข้อมูล (Feature Importance)
print("--- [Explainable AI: Feature Importance] ---")
importance = model.feature_importances_
for f, imp in zip(X.columns, importance):
    print(f"   {f}: มีความสำคัญ {imp*100:.2f}%")
```

ส่วนเสริม: ch02_ml_intelligence / ex07_xai_deep.py

.py / .ipynb

```
# ex07_xai_deep.py
#    ระดับ: Advanced
#    ภารกิจ: เจาะลึกสมอง AI ด้วย xAI (Explainable AI) และ Feature Importance

import pandas as pd
from sklearn.ensemble import RandomForestClassifier

# 1. โหลดข้อมูล
df = pd.read_csv('../../datasets/crop_data_v2.csv')
X = df[['temp', 'humidity', 'soil_moist', 'fert', 'light']]
y = df['label']

# 2. ฝึกฝนโมเดล
rf = RandomForestClassifier(n_estimators=100)
rf.fit(X, y)

# 3. เจาะลึกความสำคัญของข้อมูล (xAI)
importance = rf.feature_importances_
feature_names = X.columns

# จัดเรียงลำดับความสำคัญจากมากไปน้อย
feature_importance = sorted(zip(feature_names, importance), key=lambda x: x[1], reverse=True)

print("--- [Explainable AI: Why does AI make these decisions?] ---")
for name, val in feature_importance:
    print(f"   {name:12}: ความสำคัญ {val*100:6.2f}% [{'#' * int(val*50)}]")
```

ส่วนเสริม: ch03_deep_learning / ex01_relu_demo.py

.py / .ipynb

```
sample_values = [-5, 12, -2, 8]
relu_output = [max(0, x) for x in sample_values]
print(f"ผลลัพธ์จาก ReLU สำหรับ {sample_values} คือ {relu_output}")
```

ส่วนเสริม: ch03_deep_learning / ex02_activation_vis.py

.py / .ipynb

```python
# ex02_activation_vis.py
#    ระดับ: Intermediate
import numpy as np
import matplotlib.pyplot as plt

def relu(x): return np.maximum(0, x)
def sigmoid(x): return 1 / (1 + np.exp(-x))

x = np.linspace(-10, 10, 100)
y_relu = relu(x)
y_sig = sigmoid(x)

plt.plot(x, y_relu, label='ReLU', color='orange')
plt.plot(x, y_sig, label='Sigmoid', color='cyan')
plt.title('Activation Functions: Comparing Relay vs Probabilities')
plt.grid(True)
plt.legend()
# plt.show()
```

ส่วนเสริม: ch03_deep_learning / ex03_brain_node.py

.py / .ipynb

```python
# ex03_brain_node.py
#    ระดับ: Advanced
#    ภารกิจ: เข้าใจการทำงานของ Neuron (Weight & Bias)

def neuron(input_val, weight, bias):
    # ผลบวกเชิงเส้น (Linear Combination)
    z = (input_val * weight) + bias
    # การเปิดใช้งาน (Activation)
    a = max(0, z) # ReLU
    return a

# ลองปรับ Weight และ Bias
w = 0.8
b = -2.0
val = 10.0

result = neuron(val, w, b)
print(f"   Input: {val} * Weight: {w} + Bias: {b} = {result}")
```

ส่วนเสริม: ch03_deep_learning / ex04_sequential_model.py

.py / .ipynb

```python
# ex04_sequential_model.py
#    ระดับ: Advanced
#    ภารกิจ: ลอจิกของ Sequential Model (Layer by Layer)

def layer(inputs, weights, biases):
    # ผลลัพธ์ของเลเยอร์ (Vectorization Mock)
    return [max(0, i * w + b) for i, w, b in zip(inputs, weights, biases)]

# Input -> Hidden Layer -> Output Layer
input_data = [1.0, 0.5, -1.2]
hidden_w = [0.8, 0.2, 0.1]
hidden_b = [0, -0.5, 0.2]

# 1. รัน Hidden Layer
h1 = layer(input_data, hidden_w, hidden_b)
print(f"   Hidden Layer Results: {h1}")

# 2. รัน Output
output_w = [0.9, 0.9, 0.9]
output_b = [0, 0, 0]
final = layer(h1, output_w, output_b)
print(f"   Final Prediction: {sum(final)}")
```

ส่วนเสริม: ch03_deep_learning / ex05_overfitting_check.py

.py / .ipynb

```python
# ex05_overfitting_check.py
#    ระดับ: Advanced
#    ภารกิจ: ความเข้าใจ Overfitting (การที่ AI จำข้อสอบมากเกินไป)

train_acc = 99.8 # แม่นยำสูงมากในห้องเรียน
test_acc = 65.2  # แม่นยำต่ำมากเมื่อเจอของจริง

print(f"   Train Accuracy: {train_acc}%")
print(f"   Test Accuracy: {test_acc}%")

# การวินิจฉัย (Diagnosis)
if train_acc - test_acc > 20:
    print("   สถานะ: พบปัญหา Overfitting! แนะนำให้เพิ่ม Dropout หรือหดเลเยอร์ลง")
else:
    print("   สถานะ: โมเดลมีคุณสมบัติที่ดี (Good Generalization)")
```

ส่วนเสริม: ch03_deep_learning / ex07_relu_activation.py

.py / .ipynb

```python
# ex07_relu_activation.py
#    ระดับ: Senior (ม.ปลาย)
#    ภารกิจ: เข้าใจการทำงานของ ReLU (Rectified Linear Unit)

import matplotlib.pyplot as plt
import numpy as np

# 1. ฟังก์ชัน ReLU: ถ้า x <= 0 คืนค่า 0, ถ้า x > 0 คืนค่า x
def relu(x):
    return np.maximum(0, x)

# 2. ทดลองสุ่มค่า Input มาช่วง -10 ถึง 10
x_values = np.linspace(-10, 10, 100)
y_values = relu(x_values)

print("--- [Deep Learning: ReLU Activation] ---")
print(f"Input: -5 -> Output: {relu(-5)}")
print(f"Input: 8  -> Output: {relu(8)}")

# (หากต้องการใช้บน Colab สามารถพล็อตภาพเพื่อดูสมองกลได้)
# plt.plot(x_values, y_values)
# plt.title("ReLU Activation Function")
# plt.show()
```

ส่วนเสริม: ch04_cv_nlp / ex01_pixel_access.py

.py / .ipynb

```python
# ex01_pixel_access.py
#    ระดับ: Intermediate
#    ภารกิจ: เข้าใจพิกัดภาพ (X, Y) และสี (R, G, B)

# สมมติว่าพิกัด 10, 10
x, y = 10, 10
color = (34, 139, 34) # พิกัดนี้มีสีเขียว (Forest Green)

print(f"   พิกัด: {x}, {y}")
print(f"   สีที่พบ: R={color[0]}, G={color[1]}, B={color[2]}")

if color[1] > color[0] and color[1] > color[2]:
    print("   ระบบสรุป: นี่คือส่วนของ 'ใบพืช' ที่ปกติดี")
else:
    print("   ระบบสรุป: พบเฉดสีผิดปกติ! กำลังส่งต่อให้ AI วิเคราะห์โรค")
```

ส่วนเสริม: ch04_cv_nlp / ex02_hsv_mask.py

.py / .ipynb

```python
# ex02_hsv_mask.py
#    ระดับ: Advanced
#    ภารกิจ: ตรรกะ HSV (Hue, Saturation, Value) เพื่อคัดกรองรอยโรค

# ช่วงสีน้ำตาล (Brown Spots)
low_h, low_s, low_v = 10, 100, 20
high_h, high_s, high_v = 20, 255, 200

# สมมติสีที่อ่านได้จากพิกเซล
current_hsv = (15, 150, 100)

print(f"   ตรวจสอบที่ HSV: {current_hsv}")

if (low_h <= current_hsv[0] <= high_h and
    low_s <= current_hsv[1] <= high_s and
    low_v <= current_hsv[2] <= high_v):
    print("   พบรอยโรค! พิกัดนี้คือเฉดสีน้ำตาล (Sick Area)")
else:
    print("   พิกัดนี้ปกติ (Healthy Area)")
```

ส่วนเสริม: ch04_cv_nlp / ex03_intent_map.py

.py / .ipynb

```python
intents = {"รดน้ำพืชหน่อย": "WATERING_ON", "อากาศเป็นยังไง": "WEATHER_REPORT"}
user_voice = "รดน้ำพืชหน่อย"
print(f"   รับเสียง: {user_voice} ->    สั่งงาน: {intents.get(user_voice)}")
```

ส่วนเสริม: ch04_cv_nlp / ex04_sentiment_basic.py

.py / .ipynb

```python
# ex04_sentiment_basic.py
#    ระดับ: Intermediate
#    ภารกิจ: วิเคราะห์อารมณ์ของผู้ใช้งาน (Sentiment Score)

good_words = ["ดี", "ชอบ", "สุดยอด", "ขอบคุณ"]
bad_words = ["พัง", "ไม่ดี", "แย่", "หยุด"]

user_text = "ขอบคุณมากครับ ระบบทำงานได้ดีมาก"
score = 0

for word in user_text.split():
    if word in good_words: score += 1
    elif word in bad_words: score -= 1

if score > 0:
    print("   AI สัมผัสถึง: อารมณ์เชิงบวก (Positive)")
elif score < 0:
    print("   AI สัมผัสถึง: อารมณ์เชิงลบ (Negative)")
else:
    print("   AI สัมผัสถึง: กลางๆ (Neutral)")
```

ส่วนเสริม: ch04_cv_nlp / ex05_bot_liaison.py

.py / .ipynb

```python
# ex05_bot_liaison.py
#    ระดับ: Advanced
#    ภารกิจ: ระบบแชทพูดคุย (Basic Chatbot Logic)

responses = {
    "สวัสดี": "สวัสดีครับ! ผม Agri-Bot มีอะไรให้ช่วยไหม?",
    "สบายดีไหม": "ผมเป็น AI ครับ สบายดีและพร้อมรดน้ำต้นไม้เสมอ!",
    "ขอบคุณ": "ยินดีเสมอครับ ขอให้ได้ผลผลิตที่ดีนะครับ"
}

def get_response(text):
    for key in responses:
        if key in text:
            return responses[key]
    return "ไม่เข้าใจครับ ผมกำลังเรียนรู้ประโยคใหม่ๆ อยู่"

# จำลองการสนทนา
print(f"   Bot: {get_response('สวัสดี')}")
print(f"   Bot: {get_response('ขอบคุณ')}")
```

ส่วนเสริม: ch04_cv_nlp / ex08_hsv_masking.py

.py / .ipynb

```python
# ex08_hsv_masking.py
#    ระดับ: Vocational/Research (ปวช./วิจัย)
#    ภารกิจ: ตัดกรองเฉดสีเพื่อตรวจหาโรคพืช (Brown Spot Detection)

# หมายเหตุ: โค้ดนี้ใช้สำหรับเข้าใจตรรกะเบื้องต้นของ OpenCV (HSV)
# ในการใช้งานจริง ต้องนำไปครอบด้วย cv2.imread(image)

print("--- [Computer Vision: HSV Masking Logic] ---")

# ช่วงสีน้ำตาล (Brown Spot): ต่ำสุด - สูงสุด
lower_brown = [10, 50, 20]
upper_brown = [30, 255, 200]

print(f"   เป้าหมาย: ตรวจหาจุดสีในช่วง {lower_brown} ถึง {upper_brown}")
print("... ระบบกำลังแยกเฉดสีน้ำตาลออกจากใบไม้สีเขียว ...")
print("   ผลลัพธ์: พบรอยโรคส่วนเกิน 15% ของพื้นที่ใบพืช")
```

ส่วนเสริม: ch04_cv_nlp / ex09_intent_mapping.py

.py / .ipynb

```python
# ex09_intent_mapping.py
#    ระดับ: Senior (ม.ปลาย)
#    ภารกิจ: แปลงประโยคภาษาธรรมชาติ ให้เป็น "คำสั่งงาน AI" (Intent Classification)

# 1. นิยามความหมายของประโยค (Intent Dictionary)
intents = {
    "รดน้ำพืชหน่อย": "WATERING_ON",
    "อากาศเป็นยังไง": "WEATHER_REPORT",
    "ช่วยดูแลทุเรียน": "MONITORING_START",
    "จบภารกิจ": "SYSTEM_SHUTDOWN"
}

# 2. จำลองรับเสียงคำพูดจากนักนวัตกรรม
user_input = "อากาศเป็นยังไง"

# 3. แมปเป็นคำสั่งให้ระบบทำงาน
action = intents.get(user_input, "UNKNOWN_COMMAND")

print(f"   เสียงผู้ใช้: {user_input}")
print(f"   คำสั่ง AI: {action}")
```

ส่วนเสริม: ch05_pygame_zero / ex01_basic_actor.py

.py / .ipynb

```python
# ex01_basic_actor.py
#    ระดับ: Beginner
import pgzrun

WIDTH = 800
HEIGHT = 600

# สร้างตัวละคร (ต้องมีรูป alien.png ใน images)
alien = Actor('alien')
alien.pos = 400, 300

def draw():
    screen.clear()
    screen.fill((0, 100, 0)) # พื้นสีเขียวเข้ม
    alien.draw()

pgzrun.go()
```

ส่วนเสริม: ch05_pygame_zero / ex02_keyboard_move.py

.py / .ipynb

```python
# ex02_keyboard_move.py
#    ระดับ: Beginner
import pgzrun

WIDTH = 800
HEIGHT = 600

harvester = Actor('harvester')
harvester.pos = 400, 300

def draw():
    screen.clear()
    screen.fill((34, 139, 34))
    harvester.draw()

def update():
    # บังคับด้วยคีย์บอร์ด
    if keyboard.left: harvester.x -= 5
    if keyboard.right: harvester.x += 5
    if keyboard.up: harvester.y -= 5
    if keyboard.down: harvester.y += 5

pgzrun.go()
```

ส่วนเสริม: ch05_pygame_zero / ex03_smart_move.py

.py / .ipynb

```python
# ex03_smart_move.py
#    ระดับ: Intermediate
import pgzrun

WIDTH = 800
HEIGHT = 600

harvester = Actor('harvester')
target_x, target_y = 600, 100

def draw():
    screen.clear()
    screen.fill((34, 139, 34))
    harvester.draw()
    # วาดจุดเป้าหมาย (จำลองพิกัดจาก AI)
    screen.draw.circle((target_x, target_y), 5, "red")

def update():
    # ตรรกะช่วยขยับ (Autonomous Logic)
    if harvester.x < target_x: harvester.x += 2
    elif harvester.x > target_x: harvester.x -= 2

    if harvester.y < target_y: harvester.y += 2
    elif harvester.y > target_y: harvester.y -= 2

pgzrun.go()
```

ส่วนเสริม: ch05_pygame_zero / ex04_collision_sensor.py

.py / .ipynb

```
# ex04_collision_sensor.py
#    ระดับ: Intermediate
import pgzrun

WIDTH = 800
HEIGHT = 600

harvester = Actor('harvester')
harvester.pos = 100, 100

plant = Actor('plant')
plant.pos = 400, 300

def draw():
    screen.clear()
    screen.fill((34, 139, 34))
    harvester.draw()
    plant.draw()

    # แสดงสถานะการชน
    if harvester.colliderect(plant):
        screen.draw.text("DETECTED: AGRI-OBJECT FOUND!", (20, 20), color="yellow")

def update():
    if keyboard.right: harvester.x += 5
    if keyboard.left: harvester.x -= 5
    if keyboard.up: harvester.y -= 5
    if keyboard.down: harvester.y += 5

pgzrun.go()
```

ส่วนเสริม: ch05_pygame_zero / ex05_ai_harvester_sim.py

.py / .ipynb

```python
# ex05_ai_harvester_sim.py
#    ระดับ: Advanced
#    ภารกิจ: รถเก็บเกี่ยวอัจฉริยะ (Full Autonomous Agent)

import pgzrun
import random

WIDTH = 800
HEIGHT = 600

harvester = Actor('harvester')
harvester.pos = 400, 300

# พืช 3 ต้นที่มีสถานะต่างกัน
plants = []
for i in range(3):
    p = Actor('plant')
    p.pos = (random.randint(100, 700), random.randint(100, 500))
    p.sick = random.choice([True, False]) # สุ่มว่าเป็นโรคหรือไม่
    plants.append(p)

def draw():
    screen.clear()
    screen.fill((34, 139, 34))
    harvester.draw()
    for p in plants:
        p.draw()
        if p.sick: screen.draw.circle(p.pos, 15, "red") # แสดงสัญลักษณ์ "เป็นโรค"

def update():
    # ค้นหาพืชที่เป็นโรคที่ใกล้ที่สุด (จำลองการตัดสินใจของ AI)
    for p in plants:
        if p.sick:
            # ขยับไปหาพืชที่เป็นโรค
            if harvester.x < p.x: harvester.x += 2
            elif harvester.x > p.x: harvester.x -= 2
            if harvester.y < p.y: harvester.y += 2
            elif harvester.y > p.y: harvester.y -= 2

            # ถ้าถึงตำแหน่ง ให้ "ทำความสะอาด" (เปลี่ยนสถานะ)
            if harvester.colliderect(p):
                p.sick = False
                print("   Mission Complete: Plant Cured!")

pgzrun.go()
```

ส่วนเสริม: ch05_pygame_zero / ex10_alien_mission.py

.py / .ipynb

```python
# ex10_alien_mission.py
#    ระดับ: Junior (ม.ต้น)
#    ภารกิจ: เริ่มต้นเขียนเกมแรกด้วย PyZero (Alien Mission)
#    วิธีรัน: พิมพ์ `pgzrun ex10_alien_mission.py` ใน Terminal

import pgzrun

# 1. ตั้งค่าหน้าจอ
WIDTH = 800
HEIGHT = 600

# 2. สร้าง Actor (ต้องมีรูป alien.png ในโฟลเดอร์ images)
alien = Actor('alien')
alien.pos = 400, 300

def draw():
    screen.clear()
    screen.fill((0, 128, 0)) # สีพื้นหลังเขียว
    alien.draw()
    screen.draw.text("My First PyZero Game!", (20, 20), color="white")

def update():
    # บังคับด้วยปุ่มลูกศร
    if keyboard.left:
        alien.x -= 5
    elif keyboard.right:
        alien.x += 5
    elif keyboard.up:
        alien.y -= 5
    elif keyboard.down:
        alien.y += 5

pgzrun.go()
```

ส่วนเสริม: ch05_pygame_zero / ex11_smart_harvester.py

.py / .ipynb

```python
# ex11_smart_harvester.py
#    ระดับ: Senior (ม.ปลาย)
#    ภารกิจ: รถเก็บเกี่ยวอัจฉริยะ (AI-Driven Harvester)
#    วิธีรัน: พิมพ์ `pgzrun ex11_smart_harvester.py` ใน Terminal

import pgzrun
import random

WIDTH = 800
HEIGHT = 600

# สร้างตัวละคร (Harvester และ พืช)
harvester = Actor('harvester')
harvester.pos = (400, 500)

plants = []
for i in range(5):
    p = Actor('plant')
    p.pos = (random.randint(100, 700), random.randint(100, 400))
    # จำลองสถานะพืช (0: ปกติ, 1: เป็นโรค)
    p.status = random.randint(0, 1)
    plants.append(p)

def draw():
    screen.clear()
    screen.fill((34, 139, 34))
    harvester.draw()
    for p in plants:
        p.draw()
        if p.status == 1:
            screen.draw.text("SICK", (p.x, p.y-20), color="red")

def update():
    # ตรรกะระดับ Senior: วิ่งไปหาพืชที่เป็นโรค (SICK) โดยอัตโนมัติ
    for p in plants:
        if p.status == 1:
            if harvester.x < p.x: harvester.x += 2
            elif harvester.x > p.x: harvester.x -= 2
            if harvester.y < p.y: harvester.y += 2
            elif harvester.y > p.y: harvester.y -= 2

pgzrun.go()
```

ก.2 คลังรหัสสมุดจด Jupyter Notebook (.ipynb)

โค้ดฉบับเต็มของแบบฝึกหัดที่จำลองบน Google Colab (ดึงเฉพาะส่วนที่เป็นคำสั่ง Python ภายในไฟล์ .ipynb)

สมุดจด: ch01_foundations / ch01_foundations_master.ipynb

.py / .ipynb

```
print("ยินดีต้อนรับสู่ Python x AI 2026!")

# --------

temp = 32.5
humidity = 65
avg = (temp + humidity) / 2
print(f"Average: {avg}")

# --------

temp = 36
if temp > 35:
    print("   เปิดพัดลม")
else:
    print("   อากาศปกติ")

# --------

sensors = [32, 35, 38, 30, 31]
for s in sensors:
    if s > 35: print(f"   จุดตรวจเจอความร้อน: {s}")

# --------
```

สมุดจด: ch01_foundations / chapter1_lab.ipynb

.py / .ipynb

```
import sys
print(f"Python Version: {sys.version}")
print("Cloud Lab Initialized: Ready to Code!")

# --------

# ระบบบันทึกชื่อนักนวัตกรรม
name = input("ระบุชื่อรหัสนักวิทย์ของคุณ: ")
print(f"ยินดีต้อนรับคุณ {name} เข้าสู่ปี 2026!")

# ระบบตรวจสอบอุณหภูมิฟาร์มอัจฉริยะ
temp = 38
if temp > 35:
    print("   แจ้งเตือน: อุณหภูมิสูงเกินไป! เปิดพัดลม...")
else:
    print("   สถานะ: อุณหภูมิปกติ")

# --------

# ข้อมูลความชื้นจาก 5 จุดในฟาร์ม
moisture_levels = [45, 32, 55, 28, 40]

for m in moisture_levels:
    status = "ปกติ" if m >= 40 else "ต่ำเกินไป (ต้องรดน้ำ)"
    print(f"จุดตรวจที่ระดับ {m}%: สถานะ {status}")

# --------
```

สมุดจด: ch02_ml_intelligence / ch02_ml_intelligence_master.ipynb

.py / .ipynb

```
import pandas as pd
df = pd.read_csv('../../datasets/crop_data_v2.csv')
df.head()

# --------

df_clean = df.fillna(df.mean())
print(f"ทำความสะอาดข้อมูลเรียบร้อย: {len(df_clean)} แถว")

# --------

from sklearn.ensemble import RandomForestClassifier
X = df_clean.drop(['label', 'yield_weight'], axis=1)
y = df_clean['label']

model = RandomForestClassifier()
model.fit(X, y)
print(f"   ทายผลผลิตจากเซนเซอร์ใหม่: {model.predict([[35, 75, 40, 15, 12]])}")

# --------
```

สมุดจด: ch02_ml_intelligence / chapter2_ml_lab.ipynb

.py / .ipynb

```
import pandas as pd
import numpy as np

# จำลองข้อมูลจากฟาร์ม 5 จุด
data = {'ความชื้น': [45, 32, np.nan, 28, 40], 'ปุ๋ย': [10, 15, 12, np.nan, 18]}
df = pd.DataFrame(data)

# ซ่อมข้อมูลแหว่งด้วยค่าเฉลี่ย (Mean)
df.fillna(df.mean(), inplace=True)
print("   ข้อมูลถูกซ่อมเรียบร้อยแล้ว!")
print(df)

# --------

from sklearn.ensemble import RandomForestClassifier

# 1. ข้อมูลตัวอย่าง [ดินดี, น้ำชุ่ม, แสงพอ]
X = [[1, 1, 1], [0, 0, 0], [1, 0, 1], [0, 1, 0]]
y = ['โตดี', 'เฉา', 'พอใช้', 'พอใช้']

# 2. สร้างโมเดลและฝึกสอน
model = RandomForestClassifier(n_estimators=100)
model.fit(X, y)

# 3. ลองให้ AI ทายค่าใหม่!
test_farm = [[1, 1, 0]] # ดินดี, น้ำชุ่ม, แสงน้อย
prediction = model.predict(test_farm)
print(f"   AI ทายว่า: {prediction[0]}")

# --------

# ค้นหาว่าตัวแปรไหนมีผลต่อการตัดสินใจที่สุด
importance = model.feature_importances_
features = ['คุณภาพดิน', 'ปริมาณน้ำ', 'แสงแดด']

for f, imp in zip(features, importance):
    print(f"   {f}: มีความสำคัญ {imp*100:.2f}%")

# --------
```

สมุดจด: ch03_deep_learning / ch03_deep_learning_master.ipynb

.py / .ipynb

```
import numpy as np
def relu(x): return np.maximum(0, x)
print(f"ReLU Input -10: {relu(-10)} | ReLU Input 10: {relu(10)}")

# --------

w, b, val = 0.8, -2.0, 10.0
z = (val * w) + b
print(f"   Linear Output: {z} -> Activated (ReLU): {max(0, z)}")

# --------

inputs = [1, 0.5]
weights = [0.8, 0.2]
hidden = max(0, sum([i*w for i,w in zip(inputs, weights)]))
print(f"   Hidden Layer Active Node: {hidden}")

# --------
```

สมุดจด: ch03_deep_learning / chapter3_dl_lab.ipynb

.py / .ipynb

```
# ReLU คือคนคัดกรอง: ถ้าเป็นค่าลบให้เป็น 0 ถ้าเป็นบวกให้ผ่านได้
sample_values = [-5, 12, -2, 8]
relu_output = [max(0, x) for x in sample_values]
print(f"ผลลัพธ์จาก ReLU สำหรับ {sample_values} คือ {relu_output}")

# --------

# จำลองการสร้าง AI สมองกล 4 เลเยอร์
def build_agri_brain():
    print("สร้าง Input Layer... (รับภาพใบไม้)")
    print("เพิ่ม ReLU Layer... (กรองข้อมูล)")
    print("เพิ่ม Dropout Layer (0.2)... (ป้องกันการจำผิด)")
    print("เพิ่ม Softmax Layer... (ทำนายว่าเป็นโรคอะไร)")

build_agri_brain()
print("   สมองดิจิทัลพร้อมเรียนรู้แล้ว!")

# --------
```

สมุดจด: ch04_cv_nlp / ch04_cv_nlp_master.ipynb

.py / .ipynb

```
color = (34, 139, 34) # เขียวปกติ
if color[1] > color[0]: print("   ใบพืชปกติดี (High Green)")

# --------

low_brown = [10, 100, 20]
pixel_hsv = (15, 150, 120)
if low_brown[0] <= pixel_hsv[0] <= 30: print("   พบเป้าหมาย: รอยโรค (Brown Spot)")

# --------

voice = "รดน้ำต้นไม้"
if "รดน้ำ" in voice: print("   สั่งการ: WATER_ON")

# --------

text = "ระบบพังอีกแล้ว แย่มาก"
bad_words = ["พัง", "แย่"]
count = sum([1 for w in bad_words if w in text])
if count > 0: print("   พบอารมณ์เชิงลบ (Negative)")

# --------
```

สมุดจด: ch04_cv_nlp / chapter4_cv_nlp_lab.ipynb

.py / .ipynb

```
import cv2
print("จำลองการใช้งาน HSV Masking และ Haar Cascades")
# face_cascade = cv2.CascadeClassifier('models/haarcascade_agri.xml')
print("   พร้อมตรวจจับวัตถุในฟาร์มแล้ว!")

# --------

intents = {"รดน้ำพืชหน่อย": "WATERING_ON", "อากาศเป็นยังไง": "WEATHER_REPORT"}
user_voice = "รดน้ำพืชหน่อย"
print(f"   รับเสียง: {user_voice} ->    สั่งงาน: {intents.get(user_voice)}")

# --------
```

สมุดจด: ch05_pygame_zero / ch05_pygame_zero_master.ipynb

.py / .ipynb

```
import pgzrun

WIDTH = 800
HEIGHT = 600

harvester = Actor('harvester')
harvester.pos = 400, 300

def draw():
    screen.clear()
    screen.fill((34, 139, 34)) # สีเขียวฟาร์ม
    harvester.draw()
    screen.draw.text("Agri-AI Simulation Mode", (20, 20), color="white", fontsize=30)

def update():
    if keyboard.space:
        harvester.x += 2

pgzrun.go()

# --------
```

ภาคผนวก ข. โค้ดเฉลยสำหรับอาจารย์ (Instructor Solutions)

ชุดรหัสคำสั่งฉบับสมบูรณ์สำหรับวิทยากรหรือผู้ช่วยสอน (TA) ซึ่งถูกรวมไว้สำหรับการตรวจสอบความถูกต้องของการทำแบบฝึกหัด

เฉลยบทเรียน: ch01_solutions.py

.py / .ipynb

```
# ch01_solutions.py
#    สำหรับวิทยากรและผู้ช่วย (TA) - เฉลยและเทคนิคบทที่ 1
#    จุดที่เด็กชอบติด (Common Pitfalls):
# 1. ลืมย่อหน้า (Indentation) หลังจากใช้ if/else
# 2. ลืมใส่เครื่องหมายโคลอน (:) ท้ายเงื่อนไข
# 3. ใส่ input() แล้วลืมแปลงเป็น int/float ก่อนคำนวณ

import sys

#    เฉลย Lab ภารกิจทักทาย (Junior)
def solve_greeting():
    print("--- [TA Solution: Greeting Bot] ---")
    name = "Student_A" # หรือใช้ input()
    print(f"สวัสดีคุณ {name}!")

#    เฉลย Lab ระบบควบคุมพัดลมอัตโนมัติ (Advanced)
def solve_smart_fan(temp, humidity):
    print(f"\n--- [TA Solution: Smart Fan Logic] ---")
    # ตรรกะ: ถ้าร้อนมาก หรือ ชื้นมาก ให้เปิดพัดลมแรง
    if temp > 35 or humidity > 80:
        return "   เปิดพัดลมระดับ 3 (แรงสุด)"
    elif temp > 30:
        return "   เปิดพัดลมระดับ 1 (เบา)"
    else:
        return "   ปิดพัดลม ประหยัดพลังงาน"

if __name__ == "__main__":
    solve_greeting()
    status = solve_smart_fan(36.5, 85)
    print(f"ผลการตัดสินใจ: {status}")
```

เฉลยบทเรียน: ch02_solutions.py

.py / .ipynb

```python
# ch02_solutions.py
#    สำหรับวิทยากรและผู้ช่วย (TA) - เฉลยและเทคนิคบทที่ 2
#    จุดที่เด็กชอบติด (Common Pitfalls):
# 1. ลืมโหลด pandas (`import pandas as pd`)
# 2. ใส่ชื่อคอลัมน์ใน X, y ผิด (เช็คสะกดตัวเล็ก/ใหญ่)
# 3. ไม่ได้ลบ NaNs ออกก่อน (`df.dropna()`)

import pandas as pd
from sklearn.ensemble import RandomForestClassifier

#    เฉลย Lab พยากรณ์ผลผลิต (Ready/Sick)
def solve_harvest_prediction():
    print("--- [TA Solution: Random Forest Model] ---")

    # 1. โหลดข้อมูลจำลอง
    df = pd.read_csv('../../datasets/crop_data_v2.csv')

    # 2. ล้างข้อมูล (สำคัญมาก!)
    df_clean = df.fillna(df.mean())
    X = df_clean[['temp', 'humidity', 'soil_moist', 'fert', 'light']]
    y = df_clean['label']

    # 3. สร้างและรันโมเดล (ใช้ป่า 100 ต้น)
    model = RandomForestClassifier(n_estimators=100)
    model.fit(X, y)

    # 4. ทดสอบทายผล
    test_sensor = [[35, 75, 40, 15, 12]]
    pred = model.predict(test_sensor)
    print(f"ผลพยากรณ์ Lab: {pred}")

if __name__ == "__main__":
    solve_harvest_prediction()
```

เฉลยบทเรียน: ch04_solutions.py

.py / .ipynb

```python
# ch04_solutions.py
#    สำหรับวิทยากรและผู้ช่วย (TA) - เฉลยและเทคนิคบทที่ 4
#    จุดที่เด็กชอบติด (Common Pitfalls):
# 1. ลืมลง OpenCV (`pip install opencv-python`)
# 2. ใส่ขอบเขตสี HSV สลับที่กัน (Low-High)
# 3. ใส่ Intent ผิด (คีย์เวิร์ดใน Dictionary ต้องเป๊ะ)

import cv2
import numpy as np

#    เฉลย Lab คัดกรองรอยโรค (HSV Masking)
def solve_hsv_masking(image):
    print("--- [TA Solution: HSV Masking Tool] ---")
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

    # ช่วงสีน้ำตาล (Brown Spot): 10-30 Hue
    lower = np.array([10, 50, 20])
    upper = np.array([30, 255, 200])

    mask = cv2.inRange(hsv, lower, upper)
    result = cv2.bitwise_and(image, image, mask=mask)
    return result

#    เฉลย Lab แชทบอทตอบคำถาม (Agri-Liaison)
def solve_agri_intent(user_voice):
    print(f"\n--- [TA Solution: NLP Intent Logic] ---")
    intents = {
        "สวัสดี": "GREETING",
        "รดน้ำ": "WATER_ON",
        "อากาศ": "WEATHER_REPORT"
    }

    for key in intents:
        if key in user_voice:
            return intents[key]
    return "UNKNOWN"

if __name__ == "__main__":
    intent = solve_agri_intent("รดน้ำต้นไม้หน่อยครับ")
    print(f"   ผลการแยกข้อความ: {intent}")
```

เฉลยบทเรียน: ch06_solutions.py

.py / .ipynb

```python
# ch06_solutions.py
#    สำหรับวิทยากรและผู้ช่วย (TA) - เฉลยและเทคนิคบทที่ 6 (Pygame Zero)
#    จุดที่เด็กชอบติด (Common Pitfalls):
# 1. เขียนชื่อไฟล์ภาพใน Actor('...') ผิด (ต้องไม่ใส่นามสกุล .png)
# 2. ลืมวางไฟล์ภาพไว้ในโฟลเดอร์ images/
# 3. ลืมเติม pgzrun.go() ท้ายไฟล์

import pgzrun

#    เฉลย Lab รถเก็บเกี่ยวอัตโนมัติ (AI Simulation Agent)
def solve_ai_agent():
    print("--- [TA Solution: Autonomous Agent Simulation] ---")

    WIDTH = 800
    HEIGHT = 600

    harvester = Actor('harvester')
    target = (600, 200) # จุดรอยโรคที่ AI ตรวจพบ

    def draw():
        screen.clear()
        harvester.draw()
        screen.draw.circle(target, 5, "red")

    def update():
        # ตรรกะการเคลื่อนที่เข้าหาเป้าหมาย (Autonomous Logic)
        if harvester.x < target[0]: harvester.x += 2
        elif harvester.x > target[0]: harvester.x -= 2

        if harvester.y < target[1]: harvester.y += 2
        elif harvester.y > target[1]: harvester.y -= 2

        # เมื่อถึงที่หมาย
        if harvester.distance_to(target) < 5:
            print("   Mission Complete: Agri-Agent Target Reached!")

    # หมายเหตุสำหรับ TA: ในเฉลยจริงต้องรันผ่าน pgzrun.go()
    # pgzrun.go()

if __name__ == "__main__":
    solve_ai_agent()
    print("TA Guide: Run with 'pgzrun ch06_solutions.py'")
```

ภาคผนวก ค. สคริปต์การนำเสนอสำหรับวิทยากร (Instructor Presentation Scripts)

คู่มือฉบับย่อสำหรับการบรรยาย (Facilitator Guide) เพื่อให้วิทยากรสามารถกำกับจังหวะการสอน (Pacing) และสร้างแรงบันดาลใจให้แก่ผู้เรียนได้อย่างมีประสิทธิภาพ

DAY 1: THE FOUNDATION & PREDICTION

Phase 1: Mission Kickoff (บทนำ)

วิทยากร: "อรุณสวัสดิ์ครับ Next-Gen Scientists ทุกคน! ยินดีต้อนรับเข้าสู่วันที่ 1 ของการเปลี่ยนโลกด้วย AI ครับ! วันนี้เราไม่ได้มานั่งเรียนทฤษฎีจดเลคเชอร์นะครับ แต่เรามา 'Co-Create' หรือสร้างนวัตกรรมร่วมขบวนไปกับ AI เพื่อนในอนาคตของเราครับ! ภารกิจของเราคือการใช้พลังของโค้ดใน Google Colab แก้ปัญหาโลกจริง ขอแค่ระเบิดพลังจินตนาการออกมาพอ!"

Phase 2: The Magic of Logic (บทที่ 1: รากฐานและตรรกะมหัศจรรย์)

วิทยากร: "น้องๆ ครับ... AI มันฉลาดได้เพราะ 'Logic' ครับ! มันคือการสั่งงานแบบ Step-by-Step มนุษย์เราคิดแบบยืดหยุ่น แต่ AI คิดแบบซื่อตรง วันนี้เราจะมาเรียนภาษาที่ AI คุยรู้เรื่องที่สุด นั่นคือ Python เราจะมาสร้าง Smart Farm Data Filter คัดกรองเฉพาะข้อมูลที่ 'อันตราย' เช่น ถ้าอุณหภูมิสูงกว่า 40 องศา ให้ AI ส่งเสียงเตือนทันที!"

Phase 3: The Fortune Teller (บทที่ 2: เครื่องจักรเรียนรู้ ML)

วิทยากร: "ยินดีต้อนรับกลับมาครับ! ช่วงบ่ายนี้เราจะสอนให้ AI 'เดาอนาคต' ครับ! นี่ไม่ใช่เวทมนตร์นะครับ แต่มันคือ Machine Learning การสอนให้หุ่นยนต์มองเห็น 'ความลับ' ที่ซ่อนอยู่ในข้อมูล! เรามีข้อมูลดิน ปุ๋ย และผลผลิตในอดีต AI จะบอกเราได้ว่าควรปลูกอะไรถึงจะกำไรที่สุด โดยใช้อัลกอริทึม Random Forest และตอกย้ำด้วยระบบ xAI (Explainable AI)"

DAY 2: DEEP DIVE & REALITY

Phase 4: Digital Brains (บทที่ 3: สมองดิจิทัล Deep Learning)

วิทยากร: "วันที่ 2 เริ่มขึ้นแล้ว! วันนี้เราจะสร้าง 'เซลล์ประสาทเทียม' เลียนแบบสมองมนุษย์กันครับ! สมองของหุ่นยนต์ Neural Network ตัวนี้จะช่วยเราทำงานที่ซับซ้อนอย่างการคัดแยกโรคพืชได้อย่างแม่นยำ"

Phase 5: Digital Eyes & Agent Communicator (บทที่ 4: ดวงตาอัจฉริยะและภาษา AI)

วิทยากร: "นักวิทยาศาสตร์มีดวงตา... แต่ AI มี 'Computer Vision' ครับ! เราจะใช้ OpenCV มาตัดกรองสี (Masking) เพื่อหา 'รอยโรคสีน้ำตาล' บนใบไม้! ใครทำได้ จะเปรียบเสมือนคุณมี X-ray ส่วนตัวเลย! และแน่นอน เราจะต้องสั่งการมันให้รู้เรื่องด้วย NLP เพื่อประมวลผลคำสั่งเชิงภาษาให้หุ่นยนต์เข้าใจ"

Phase 6: Digital Twin Simulation (บทที่ 5: โลกเสมือน Pygame Zero)

วิทยากร: "เอาล่ะครับทุกคน! โค้ดที่เราเขียนมาทั้งหมด ไม่ได้มีไว้แค่ให้มันวิ่งเป็นตัวหนังสือในจอคอมพิวเตอร์ แต่วันนี้เราจะ 'ถ่ายทอดจิตวิญญาณ' ของโมเดลเหล่านั้นลงใน **โลกเสมือนจริง (Digital Twin)** ครับ! เตรียมพบกับมิติใหม่ที่คุณจะใช้รหัส Python ประกอบร่างเข้ากับอินเทอร์เฟสเกม เพื่อบังคับ 'รถเก็บเกี่ยวอัจฉริยะ' ขับเคลื่อนฝ่าฟาร์มสามมิติ นี่คือปรากฏการณ์ใหม่ของการจำลองโลกเกษตรแห่งอนาคต ลุยเลย!"

Phase 7: Agri-Hackathon Final Pitch (บทที่ 6: การบูรณาการ)

วิทยากร: "ภารกิจสุดท้าย! ประกอบร่างทุกอย่างเข้าด้วยกัน และสวมวิญญาณ AI-Next Scientist ในตัวคุณ ใช้พี่ยักษ์ Antigravity ค้นหาข้อมูลเชิงลึกด้วย RAG และสร้างระบบการเกษตรอัจฉริยะของคุณเลย! ขอแสดงความยินดีกับทุกคนครับ อนาคตอยู่ในการ

ควบคุมด้วยโค้ดของคุณแล้ว!"

ดัชนีและอภิธานศัพท์ (Glossary & Index)

ดัชนีคำค้นเรียงตามลำดับตัวอักษรภาษาอังกฤษที่ปรากฏในคู่มือ:

- **AI-Next Scientist**: นักวิทยาศาสตร์หรือนักนวัตกรรมรุ่นใหม่ที่ใช้ AI เป็นผู้ช่วยในการทำงาน (Co-Pilot)
- **Computer Vision**: ระบบคอมพิวเตอร์วิทัศน์ที่สอนให้เครื่องจักรเข้าใจภาพและวิดีโอ (เช่น การใช้ OpenCV)
- **Deep Learning**: การเรียนรู้เชิงลึกที่เลียนแบบโครงข่ายประสาทของมนุษย์ (Neural Networks)
- **Digital Twin**: การจำลองสภาพแวดล้อมหรือเครื่องจักรจากโลกจริงเข้าสู่โลกเสมือน (เช่น การใช้ Pygame Zero จำลองฟาร์ม)
- **Dropout Layer**: การสุ่มปิดเซลล์ประสาทในโมเดล Deep Learning เพื่อป้องกันการจำข้อสอบ (Overfitting)
- **Feature Importance**: การพิจารณาใน Explainable AI ว่าปัจจัยใด (เช่น อุณหภูมิ, แสง) มีผลต่อการพยากรณ์มากที่สุด
- **Haar Cascades**: โมเดลคลาสสิฟายเออร์สำเร็จรูปที่ใช้สำหรับตรวจจับวัตถุ เช่น ใบหน้ามนุษย์ หรือใบพืช
- **HSV (Hue, Saturation, Value)**: ระบบสีที่แยกความสว่างออกจากสีอย่างชัดเจน นิยมใช้ในการวิเคราะห์ภาพถ่ายทางการเกษตร
- **Machine Learning**: การป้อนข้อมูลให้ระบบคอมพิวเตอร์เรียนรู้รูปแบบและสร้างกฎขึ้นมาด้วยตนเองเพื่อพยากรณ์ผลลัพธ์
- **NLP (Natural Language Processing)**: การประมวลผลภาษาธรรมชาติ เพื่อให้คอมพิวเตอร์ทำความเข้าใจข้อความหรือเสียงพูด
- **Random Forest**: อัลกอริทึมต้นไม้ตัดสินใจหลายๆ ต้นที่ร่วมกันวิเคราะห์และโหวตหาผลลัพธ์ที่แม่นยำที่สุด
- **ReLU (Rectified Linear Unit)**: ฟังก์ชันกระตุ้น (Activation Function) ยอดนิยมที่คืนค่าศูนย์สำหรับอินพุตติดลบและคืนค่าเดิมสำหรับบวก
- **Softmax**: ฟังก์ชันชั้นสุดท้ายในโมเดล Neural Network ที่ใช้คำนวณอัตราความน่าจะเป็นของแต่ละคลาส
- **xAI (Explainable AI)**: ปัญญาประดิษฐ์ที่สามารถอธิบายเหตุผลของการตัดสินใจของมันได้